LAPORAN SOFTWARE TESTING
CODE REVIEW MENGGUNAKAN DATASET CROP

1. Pendahuluan

1.1 Latar Belakang

Code Review merupakan proses pemeriksaan perubahan kode program yang dilakukan untuk membantu menemukan kesalahan, meningkatkan kualitas perangkat lunak, dan memastikan perubahan yang dibuat sesuai dengan tujuan pengembangannya. Dalam pengembangan perangkat lunak, proses ini dapat melibatkan pemeriksaan source code dan diskusi antara pengembang dengan reviewer sebelum perubahan diterima ke dalam proyek.

Pelaksanaan Code Review pada proyek nyata dapat dipelajari melalui dataset yang merekam perubahan kode dan diskusi review. Salah satu dataset tersebut adalah CROP (*Code Review Open Platform*), yang diperkenalkan oleh Matheus Paixao, Jens Krinke, Donggyun Han, dan Mark Harman pada tahun 2018 melalui paper *CROP: Linking Code Reviews to Source Code Changes*.

CROP menyediakan data Code Review yang dihubungkan dengan versi source code pada saat proses review berlangsung. Dengan demikian, dataset ini dapat digunakan untuk memeriksa perubahan kode dan mempelajari konteks diskusi yang menyertainya.

Pada tugas ini, proses Code Review dilakukan menggunakan dataset CROP dari proyek testrunner, yaitu proyek berbasis Python. Pemeriksaan diarahkan pada satu review yang dipilih berdasarkan ketersediaan metadata, diskusi, dan source code yang dapat diperiksa.

1.2 Tujuan

Tujuan pelaksanaan tugas ini adalah:

1. Memahami penggunaan dataset CROP sebagai sumber data untuk Code Review.
2. Mengidentifikasi satu perubahan kode berdasarkan metadata review.
3. Memeriksa perubahan source code sebelum dan sesudah revisi.
4. Mengidentifikasi potensi masalah berdasarkan hasil pemeriksaan kode.
5. Menyusun temuan, dampak, dan rekomendasi perbaikan berdasarkan bukti yang ditemukan.

6. Mendokumentasikan proses Code Review menggunakan screenshot dan penjelasan tertulis.

2. Deskripsi Dataset

2.1 Sumber Dataset

Dataset yang digunakan berasal dari CROP (*Code Review Open Platform*), yang diperkenalkan dalam paper *CROP: Linking Code Reviews to Source Code Changes* oleh Matheus Paixao, Jens Krinke, Donggyun Han, dan Mark Harman pada tahun 2018.

CROP menghubungkan data Code Review dengan versi lengkap source code yang berkaitan dengan revisi tertentu. Data yang disediakan mencakup metadata review, diskusi selama proses review, serta akses ke snapshot source code sebelum dan sesudah perubahan.

Sumber yang digunakan:

- Paper CROP: https://doi.org/10.1145/3196398.3196466
- Situs resmi CROP: https://crop-repo.github.io/
- Arsip dataset: https://zenodo.org/records/3599150

2.2 Proyek yang Dipilih

Proyek yang digunakan adalah testrunner, yang tercatat sebagai salah satu proyek dalam dataset CROP. Berdasarkan paper, proyek ini menggunakan bahasa pemrograman Python.

Pemilihan proyek ini bertujuan membatasi ruang lingkup pemeriksaan pada satu proyek sehingga perubahan kode dapat diperiksa secara lebih terarah.

2.3 Data yang Digunakan

Data awal yang digunakan adalah file testrunner.csv, yang berisi metadata revisi. Metadata tersebut digunakan untuk mengidentifikasi review dan memperoleh informasi yang diperlukan untuk menelusuri source code terkait.

Data tambahan yang dibutuhkan meliputi:

1. **Metadata review**, untuk mengetahui identitas review, revisi, status, dan commit terkait.
2. **File diskusi review**, untuk membaca deskripsi perubahan serta komentar reviewer.
3. **Source code sebelum perubahan**, untuk memahami kondisi kode awal.
4. **Source code sesudah perubahan**, untuk mengetahui perubahan yang diajukan.

Data tambahan tersebut akan diperiksa dan dicocokkan sebelum analisis dilakukan.

3. Metodologi Code Review

3.1 Pendekatan Pemeriksaan

Pendekatan yang digunakan adalah pemeriksaan kode secara manual (*manual Code Review*) dengan memanfaatkan metadata, diskusi, dan perubahan source code dari dataset CROP.

Pemeriksaan dilakukan dengan membandingkan kode sebelum dan sesudah revisi, memahami tujuan perubahan berdasarkan deskripsi review, serta memeriksa kemungkinan masalah pada implementasi yang berubah.

Pemeriksaan difokuskan pada aspek berikut:

1. **Logika program**, untuk memeriksa apakah implementasi sesuai dengan tujuan perubahan.
2. **Penanganan kondisi**, untuk mengidentifikasi kondisi batas atau kondisi tertentu yang belum ditangani.
3. **Potensi kesalahan**, untuk mengidentifikasi kesalahan implementasi yang didukung oleh bukti kode.
4. **Keterbacaan dan pemeliharaan kode**, untuk menilai apakah perubahan membuat kode lebih sulit dipahami atau dipelihara.
5. **Kesesuaian perubahan**, untuk memastikan perubahan yang diajukan konsisten dengan deskripsi dan konteks review.

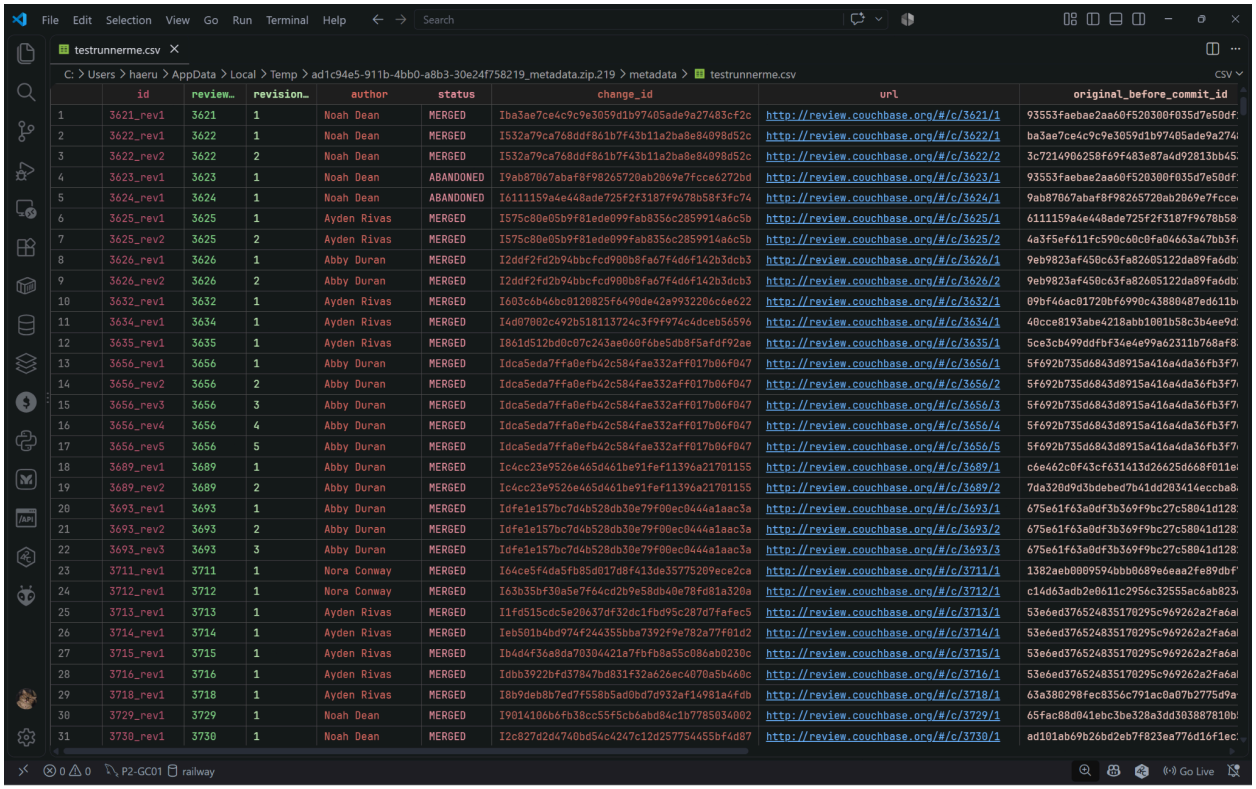
Aspek yang tidak relevan dengan perubahan yang diperiksa tidak akan dipaksakan menjadi temuan.

3.2 Tahapan Pelaksanaan

3.2.1 Identifikasi Review

Tahap pertama dilakukan dengan memeriksa metadata pada testrunner.csv untuk menemukan review yang dapat ditelusuri lebih lanjut. Kandidat review dipilih berdasarkan ketersediaan identitas revisi, status review, diskusi, dan source code yang berkaitan.

Dokumentasi:

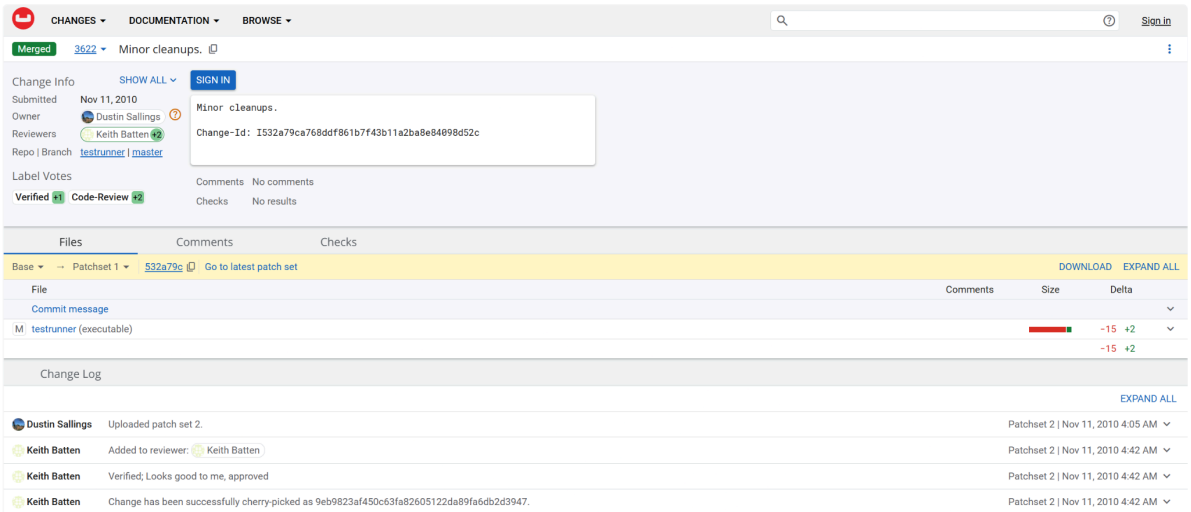


3.2.2 Pemeriksaan Diskusi Review

Setelah kandidat ditentukan, file diskusi diperiksa untuk memahami deskripsi perubahan, tujuan pengajuan kode, komentar reviewer, dan tanggapan pengembang jika tersedia.

Informasi tersebut digunakan untuk memahami konteks perubahan sebelum memeriksa implementasinya.

Dokumentasi:



Pemeriksaan diskusi dilakukan pada Review #3622 dalam proyek `testrunner`. Berdasarkan halaman review, perubahan diajukan oleh Dustin Sallings pada 11 November 2010 dengan judul "Minor cleanups". Perubahan tersebut berstatus *Merged* dan ditujukan ke branch `master`.

Deskripsi yang tersedia hanya menyebutkan "Minor cleanups", sehingga tujuan teknis perubahan belum dapat dipastikan dari deskripsi tersebut saja. Pada bagian komentar, halaman menampilkan keterangan "No comments". Dengan demikian, tidak terdapat komentar diskusi yang dapat dianalisis pada tampilan tersebut.

Meskipun tidak terdapat komentar diskusi, bagian *Change Log* menunjukkan bahwa perubahan telah diverifikasi dan disetujui oleh Keith Batten dengan keterangan "Looks good to me, approved". Halaman tersebut juga menunjukkan bahwa perubahan berhasil di-*cherry-pick* ke commit lain.

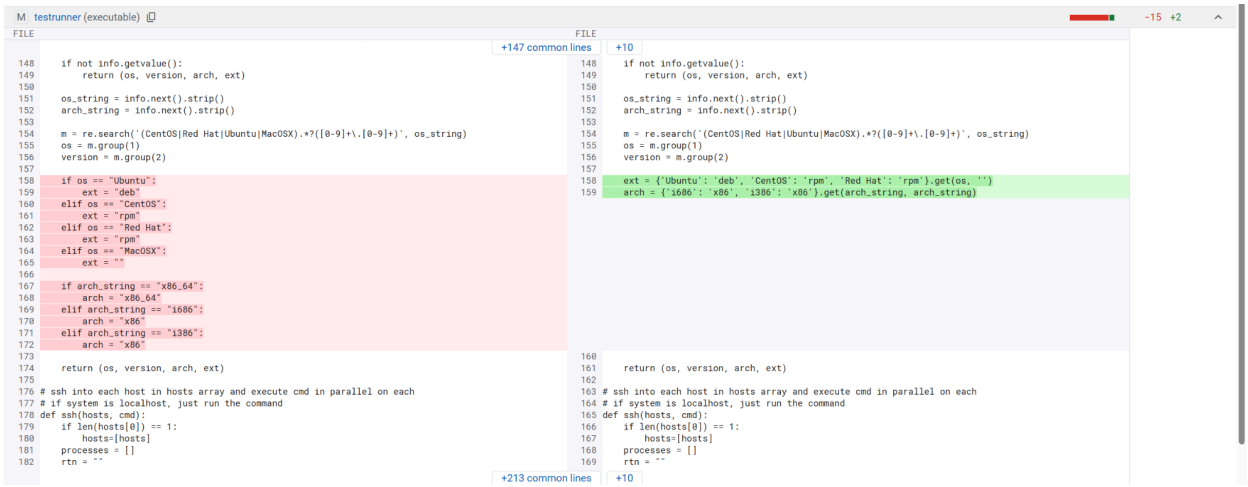
Informasi ini digunakan untuk memahami status dan konteks awal perubahan. Namun, persetujuan reviewer dan status *Merged* tidak secara otomatis membuktikan bahwa kode bebas dari kesalahan. Oleh karena itu, pemeriksaan kode sumber tetap diperlukan untuk mengevaluasi implementasi perubahan.

3.2.3 Pemeriksaan Source Code

Source code sebelum dan sesudah revisi diperiksa untuk mengidentifikasi bagian yang berubah. Pemeriksaan dilakukan dengan membandingkan implementasi sebelumnya dengan implementasi yang diajukan dalam revisi.

Perhatian diberikan pada perubahan logika, kondisi percabangan, penanganan nilai, pemanggilan fungsi, serta aspek lain yang relevan dengan kode yang diperiksa.

Dokumentasi:



Pemeriksaan kode sumber dilakukan dengan membandingkan perubahan sebelum dan sesudah revisi pada Review #3622. Berdasarkan tampilan *code diff*, perubahan dilakukan pada bagian kode yang menentukan ekstensi paket berdasarkan sistem operasi dan menormalisasi nama arsitektur.

Pada kode sebelumnya, pemilihan ekstensi paket dilakukan menggunakan beberapa kondisi if dan elif. Normalisasi arsitektur juga menggunakan serangkaian kondisi untuk mengubah nilai i686 dan i386 menjadi x86. Pada kode revisi, kedua bagian tersebut disederhanakan dengan menggunakan struktur data dictionary dan metode .get().

Perubahan ini mengurangi jumlah baris kode dan menyederhanakan cara pemetaan nilai. Dictionary pertama digunakan untuk menentukan ekstensi paket, sedangkan dictionary kedua digunakan untuk menormalisasi arsitektur. Apabila nilai arsitektur tidak ditemukan dalam dictionary, kode mempertahankan nilai aslinya.

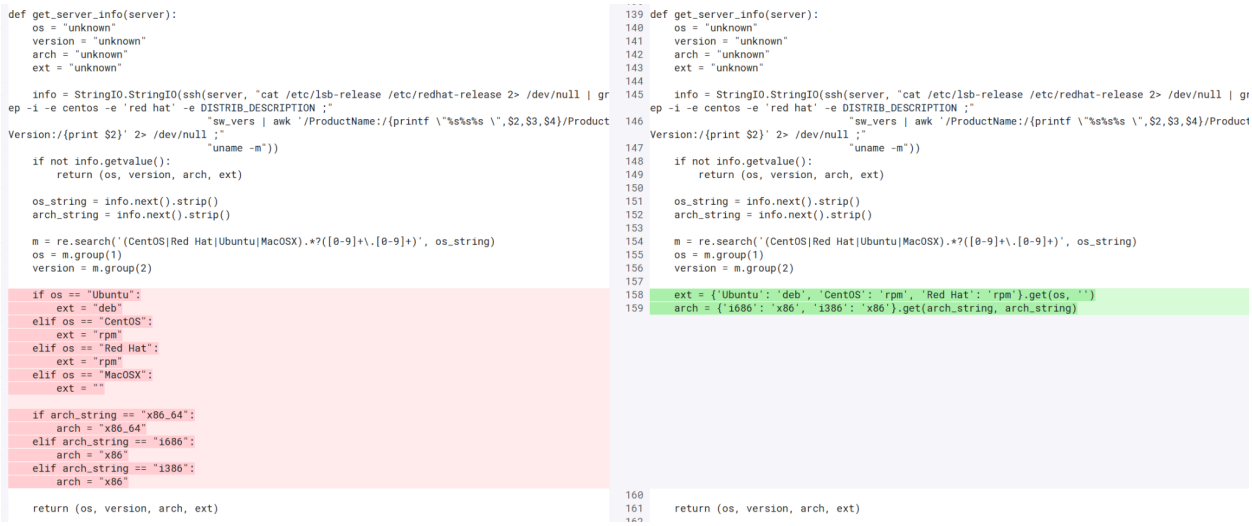
Berdasarkan potongan kode yang tersedia, perubahan utama berfokus pada penyederhanaan implementasi pemetaan nilai. Namun, pemeriksaan terhadap keseluruhan fungsi dan perilaku program masih diperlukan sebelum dapat menyimpulkan apakah perubahan tersebut mempertahankan seluruh perilaku sebelumnya atau menimbulkan potensi masalah.

3.2.4 Identifikasi Temuan

Setelah perubahan dipahami, setiap bagian yang berpotensi menimbulkan masalah dicatat sebagai kandidat temuan. Setiap temuan harus disertai bukti berupa bagian kode atau konteks diskusi yang relevan.

Temuan kemudian dijelaskan berdasarkan jenis masalah, dampak yang mungkin terjadi, dan rekomendasi perbaikannya. Jika tidak ditemukan masalah pada bagian yang diperiksa, hasil tersebut dicatat secara proporsional tanpa mengasumsikan adanya bug.

Dokumentasi:



```
def get_server_info(server):
    os = "unknown"
    version = "unknown"
    arch = "unknown"
    ext = "unknown"

    info = StringIO.StringIO(ssh(server, "cat /etc/lsb-release /etc/redhat-release 2> /dev/null | gr
ep -i -e centos -e 'red hat' -e DISTRIB_DESCRIPTION ;"
                                 "sw_vers | awk '/ProductName:/{printf \"%s%s%s \",$2,$3,$4}/Product
Version:/{print $2}' 2> /dev/null ;"
                                 "uname -m"))
    if not info.getvalue():
        return (os, version, arch, ext)

    os_string = info.next().strip()
    arch_string = info.next().strip()

    m = re.search('(CentOS|Red Hat|Ubuntu|MacOSX).*?([0-9]+\.[0-9]+)', os_string)
    os = m.group(1)
    version = m.group(2)

    if os == "Ubuntu":
        ext = "deb"
    elif os == "CentOS":
        ext = "rpm"
    elif os == "Red Hat":
        ext = "rpm"
    elif os == "MacOSX":
        ext = ""

    if arch_string == "x86_64":
        arch = "x86_64"
    elif arch_string == "i686":
        arch = "x86"
    elif arch_string == "i386":
        arch = "x86"

    return (os, version, arch, ext)
```

```
139 def get_server_info(server):
140     os = "unknown"
141     version = "unknown"
142     arch = "unknown"
143     ext = "unknown"
144
145     info = StringIO.StringIO(ssh(server, "cat /etc/lsb-release /etc/redhat-release 2> /dev/null | gr
    ep -i -e centos -e 'red hat' -e DISTRIB_DESCRIPTION ;"
146                                  "sw_vers | awk '/ProductName:/{printf \"%s%s%s \",$2,$3,$4}/Product
    Version:/{print $2}' 2> /dev/null ;"
147                                  "uname -m"))
148     if not info.getvalue():
149         return (os, version, arch, ext)
150
151     os_string = info.next().strip()
152     arch_string = info.next().strip()
153
154     m = re.search('(CentOS|Red Hat|Ubuntu|MacOSX).*?([0-9]+\.[0-9]+)', os_string)
155     os = m.group(1)
156     version = m.group(2)
157
158     ext = {'Ubuntu': 'deb', 'CentOS': 'rpm', 'Red Hat': 'rpm'}.get(os, '')
159     arch = {'i686': 'x86', 'i386': 'x86'}.get(arch_string, arch_string)
160
161     return (os, version, arch, ext)
```

Pemeriksaan perubahan kode sumber dilakukan dengan membandingkan implementasi sebelum dan sesudah perubahan pada Review #3622 dalam proyek testrunner. Berdasarkan hasil pemeriksaan, perubahan dilakukan pada fungsi get_server_info(), yang bertanggung jawab memperoleh informasi sistem operasi, versi, arsitektur, dan ekstensi paket instalasi server.

Pada implementasi sebelumnya, pemetaan ekstensi paket dan arsitektur dilakukan menggunakan beberapa kondisi if dan elif. Pada implementasi yang baru, logika tersebut disederhanakan dengan menggunakan struktur data *dictionary* dan metode .get(). Perubahan ini mengurangi jumlah baris kode serta membuat pemetaan nilai lebih ringkas.

Hasil perbandingan menunjukkan bahwa pemetaan untuk Ubuntu, CentOS, Red Hat, MacOSX, serta arsitektur i686, i386, dan x86_64 secara umum mempertahankan hasil yang sama. Namun, terdapat perbedaan pada penanganan arsitektur yang tidak tercantum secara eksplisit. Implementasi sebelumnya menggunakan nilai awal unknown apabila arsitektur tidak termasuk dalam kondisi yang ditentukan, sedangkan implementasi baru mempertahankan nilai arsitektur yang diterima.

Perbedaan tersebut dicatat sebagai aspek yang perlu diverifikasi lebih lanjut untuk memastikan kesesuaiannya dengan kebutuhan program. Berdasarkan potongan kode yang diperiksa, belum dapat dipastikan bahwa perbedaan tersebut menimbulkan kesalahan karena diperlukan informasi tambahan mengenai arsitektur yang didukung dan ketersediaan paket instalasinya.

Dengan demikian, perubahan pada Review #3622 terutama menyederhanakan implementasi pemetaan nilai. Pemeriksaan ini mengidentifikasi satu perbedaan perilaku yang perlu ditelusuri lebih lanjut, tetapi belum menghasilkan bukti yang cukup untuk menyatakan bahwa perubahan tersebut menimbulkan bug.

3.2.5 Penyusunan Hasil

Tahap terakhir adalah menyusun hasil pemeriksaan dalam tabel temuan Code Review. Hasil tersebut memuat bagian kode yang diperiksa, temuan, dampak, rekomendasi, dan bukti pendukung.

Berdasarkan pemeriksaan perubahan kode pada Review #3622, ditemukan perubahan implementasi pada fungsi get_server_info(). Perubahan tersebut menyederhanakan pemetaan sistem operasi dan arsitektur dengan mengganti beberapa kondisi if dan elif menggunakan struktur data *dictionary* dan metode .get().

Hasil pemeriksaan dirangkum dalam tabel berikut.

No.	Aspek yang Diperiksa	Hasil Pemeriksaan	Kategori Temuan
1	Penyederhanaan kode	Kondisi pemetaan diganti dengan dictionary dan metode .get(), sehingga implementasi menjadi lebih ringkas.	Perubahan implementasi
2	Pemetaan ekstensi sistem operasi	Pemetaan untuk Ubuntu (deb), CentOS (rpm), Red Hat (rpm), dan MacOSX (string kosong) tetap menghasilkan nilai yang sama.	Tidak ditemukan perbedaan pada kasus yang diperiksa
3	Pemetaan arsitektur yang dikenal	Pemetaan i686 dan i386 menjadi x86, serta penanganan x86_64, tetap menghasilkan nilai yang sama.	Tidak ditemukan perbedaan pada kasus yang diperiksa
4	Penanganan arsitektur yang tidak dikenal	Implementasi lama mempertahankan nilai unknown, sedangkan implementasi baru mempertahankan nilai arsitektur yang diterima. Dampak perbedaan ini belum diverifikasi.	Potensi masalah yang memerlukan pemeriksaan lanjutan

Berdasarkan hasil tersebut, perubahan yang paling jelas adalah penyederhanaan implementasi. Perbedaan perilaku pada penanganan arsitektur yang tidak dikenal menjadi aspek yang perlu ditelusuri lebih lanjut, khususnya untuk memastikan apakah nilai tersebut dapat digunakan dalam pembentukan URL paket instalasi.

Namun, pemeriksaan ini belum membuktikan adanya bug pada perubahan yang dilakukan. Oleh karena itu, temuan nomor empat dikategorikan sebagai potensi masalah, bukan bug terkonfirmasi. Kesimpulan akhir perlu mempertimbangkan kebutuhan sistem dan perilaku program ketika menerima arsitektur yang tidak tercantum dalam pemetaan.

4. Hasil dan Pembahasan Code Review

4.1 Identitas Review

Review yang dipilih berasal dari dataset CROP pada proyek testrunner yang menggunakan bahasa pemrograman Python. Identitas review diperoleh dari halaman review pada repositori Couchbase.

Informasi	Keterangan
Nama proyek	testrunner
Bahasa pemrograman	Python
Nomor review	3622
Nomor revisi	Patch Set 1 dan Patch Set 2
Status review	Merged

File yang diperiksa	testrunner (executable)
Tujuan perubahan	Minor cleanups (perapian kode)

4.2 Pemeriksaan Deskripsi dan Diskusi Review

Pemeriksaan diskusi dilakukan pada Review #3622 dalam proyek testrunner. Berdasarkan halaman review, perubahan diajukan oleh Dustin Sallings pada 11 November 2010 dengan judul "Minor cleanups". Perubahan tersebut berstatus *Merged* dan ditujukan ke branch master.

Deskripsi perubahan hanya menyebutkan "Minor cleanups", sehingga tujuan teknis perubahan tidak dijelaskan secara terperinci. Pada bagian komentar, halaman menampilkan keterangan "No comments". Oleh karena itu, tidak terdapat komentar diskusi yang dapat dianalisis pada tampilan tersebut.

Meskipun tidak terdapat komentar diskusi, bagian *Change Log* menunjukkan bahwa perubahan telah diverifikasi dan disetujui oleh Keith Batten dengan keterangan "Looks good to me, approved". Halaman tersebut juga mencatat bahwa perubahan berhasil di-*cherry-pick* ke commit lain.

Informasi tersebut memberikan konteks awal bahwa perubahan telah melalui proses review dan memperoleh persetujuan. Namun, status *Merged* dan persetujuan reviewer tidak secara otomatis membuktikan bahwa kode bebas dari kesalahan. Oleh sebab itu, pemeriksaan terhadap perubahan kode sumber tetap diperlukan.

4.3 Pemeriksaan Perubahan Source Code

Pemeriksaan source code dilakukan dengan membandingkan kode sebelum dan sesudah perubahan pada Review #3622. Berdasarkan hasil pemeriksaan, perubahan dilakukan pada fungsi get_server_info(), yang bertanggung jawab memperoleh informasi sistem operasi, versi, arsitektur, dan ekstensi paket instalasi server.

Pada implementasi sebelumnya, pemetaan ekstensi paket dan arsitektur dilakukan menggunakan beberapa kondisi if dan elif. Pada implementasi baru, logika tersebut disederhanakan dengan menggunakan struktur data *dictionary* dan metode .get(). Perubahan ini mengurangi jumlah baris kode dan membuat pemetaan nilai lebih ringkas.

Hasil perbandingan menunjukkan bahwa pemetaan untuk Ubuntu, CentOS, Red Hat, dan MacOSX menghasilkan nilai ekstensi yang sama pada kasus yang diperiksa. Demikian pula, pemetaan arsitektur i686 dan i386 menjadi x86, serta penanganan x86_64, tetap menghasilkan nilai yang sama.

Namun, terdapat perbedaan pada penanganan arsitektur yang tidak tercantum secara eksplisit. Implementasi sebelumnya mempertahankan nilai awal unknown apabila arsitektur tidak termasuk dalam kondisi yang ditentukan, sedangkan implementasi baru mempertahankan nilai arsitektur yang diterima. Perbedaan tersebut perlu diperiksa lebih lanjut untuk mengetahui apakah nilai yang tidak dikenal dapat digunakan dalam pembentukan URL paket instalasi.

Berdasarkan potongan kode yang diperiksa, perubahan utama berfokus pada penyederhanaan implementasi pemetaan nilai. Belum terdapat bukti yang cukup untuk menyimpulkan bahwa perbedaan penanganan arsitektur tersebut menimbulkan kesalahan. Verifikasi lebih lanjut diperlukan untuk mengetahui apakah perilaku baru tersebut sesuai dengan kebutuhan program.

4.4 Hasil Temuan Code Review

Berdasarkan pemeriksaan perubahan kode, ditemukan satu perbedaan perilaku yang perlu diverifikasi lebih lanjut. Hasil pemeriksaan dirangkum dalam tabel berikut.

No.	Bagian kode	Temuan	Dampak	Rekomendasi
1	Fungsi get_server_info(), pemetaan arsitektur	Implementasi baru mempertahankan nilai arsitektur yang diterima apabila nilainya tidak tercantum dalam dictionary. Implementasi sebelumnya menggunakan nilai awal unknown untuk arsitektur yang tidak ditangani secara eksplisit.	Perbedaan tersebut dapat memengaruhi nilai arsitektur yang digunakan ketika membentuk URL paket instalasi. Dampak aktualnya belum terverifikasi.	Periksa daftar arsitektur yang didukung dan lakukan pengujian terhadap arsitektur yang dikenal maupun yang tidak dikenal untuk memastikan perilaku yang diharapkan.

Selain temuan tersebut, pemeriksaan menunjukkan bahwa penggunaan dictionary dan metode .get() menyederhanakan pemetaan sistem operasi dan arsitektur. Pada kasus yang diperiksa,

pemetaan ekstensi paket untuk Ubuntu, CentOS, Red Hat, dan MacOSX serta pemetaan arsitektur i686, i386, dan x86_64 tetap menghasilkan nilai yang sama.

Temuan pada tabel dikategorikan sebagai potensi masalah yang memerlukan verifikasi, bukan bug terkonfirmasi. Kesimpulan tersebut diambil karena belum tersedia bukti pengujian yang menunjukkan bahwa perbedaan perilaku tersebut menyebabkan kegagalan program.

4.5 Perbandingan dengan Diskusi Reviewer

Hasil pemeriksaan manual dibandingkan dengan informasi pada halaman review asli untuk mengetahui apakah perbedaan perilaku yang ditemukan juga dibahas oleh reviewer.

No.	Temuan pemeriksaan manual	Komentar reviewer asli	Hasil perbandingan
1	Terdapat perbedaan penanganan arsitektur yang tidak dikenal antara implementasi lama dan baru.	Tidak tersedia komentar diskusi pada halaman review yang diperiksa.	Tidak dapat dibandingkan secara langsung karena tidak terdapat komentar reviewer yang membahas aspek tersebut.

Berdasarkan halaman review yang diperiksa, perubahan memperoleh persetujuan dengan keterangan "Looks good to me, approved". Namun, halaman tersebut tidak menampilkan komentar diskusi yang menjelaskan alasan persetujuan atau membahas penanganan arsitektur yang tidak dikenal.

Dengan demikian, tidak dapat dipastikan apakah perbedaan perilaku yang ditemukan dalam pemeriksaan manual juga telah dipertimbangkan oleh reviewer asli. Hasil pemeriksaan manual dan informasi dari review asli tetap dipisahkan agar tidak menimbulkan kesimpulan yang tidak didukung oleh bukti.

5. Kesimpulan

Code review pada tugas ini dilakukan menggunakan dataset CROP dengan memilih proyek testrunner yang menggunakan bahasa pemrograman Python. Review #3622 dipilih sebagai objek pemeriksaan dengan status *Merged* dan deskripsi perubahan "Minor cleanups".

Pemeriksaan dilakukan melalui identifikasi metadata, penelaahan deskripsi dan aktivitas review, serta perbandingan source code sebelum dan sesudah perubahan. Hasil pemeriksaan menunjukkan bahwa perubahan utama berupa penyederhanaan pemetaan sistem operasi dan arsitektur menggunakan dictionary dan metode .get().

Pada kasus yang diperiksa, pemetaan ekstensi paket dan arsitektur yang dikenal mempertahankan hasil yang sama. Namun, ditemukan perbedaan pada penanganan arsitektur yang tidak tercantum secara eksplisit. Perbedaan tersebut berpotensi memengaruhi pembentukan URL paket instalasi, tetapi dampak aktualnya belum dapat dipastikan tanpa pemeriksaan lanjutan dan pengujian.

Perbandingan dengan halaman review asli menunjukkan bahwa perubahan telah memperoleh persetujuan, tetapi tidak tersedia komentar diskusi yang dapat digunakan untuk membandingkan temuan manual dengan pembahasan reviewer. Oleh karena itu, hasil code review ini menekankan pentingnya memeriksa perubahan implementasi berdasarkan bukti kode dan membedakan potensi masalah dari bug yang telah terkonfirmasi.